Image Processing for Mechatronics MCTR1010
Mechatronics Engineering
Faculty of Engineering and Materials Science
German University in Cairo

Goalkeeper Robotic Arm

By

Team 21

Besher Hezzeni 55-22381
Mohamed Lotfy 55-1400
Mazen Amr 55-0983
Michael Hany 55-23395
Youssef Elkholy 55-1115
Mariam Ahmed 55-4057

Course Team

Assoc. Prof. Dr. Omar M.Shehata

April 16, 2026

Chapter 1

Literature Review

Vision-based robotic systems have become an essential research area in mechatronics and robotics, particularly for applications that require interaction with dynamic environments. One of the key applications is robotic goalkeeping, where the robot must detect a moving ball, estimate its trajectory, and respond rapidly to intercept it. Image processing techniques combined with control algorithms allow robots to perform such tasks with increasing accuracy and speed. Several studies have explored different approaches to ball detection, trajectory prediction, and robotic actuation using camera-based sensing systems.

Ball detection and tracking is the first step in implementing a vision-based goalkeeper system. Kim et al. [1] proposed a real-time ball tracking system using classical computer vision techniques implemented in OpenCV. Their method relies on color segmentation to isolate the ball from the background by converting the captured image into the HSV color space. After segmentation, contour detection and centroid extraction are used to determine the exact position of the ball in each frame. The system was able to track a moving ball efficiently in real time, making it suitable for robotic applications where fast reaction times are required. The study demonstrated that lightweight image processing techniques can be effectively implemented on embedded platforms such as Raspberry Pi for real-time object tracking.

In addition to detecting the ball position, predicting its future trajectory is critical for robotic goalkeeper systems. Liu et al. [2] introduced a vision-based trajectory prediction approach using a Kalman filtering algorithm. In this method, the ball's position and velocity are estimated from consecutive camera frames, and a predictive model is used to estimate the future path of the ball. By predicting the trajectory in advance, the robot can move proactively rather than reacting only after the ball reaches a certain location. The results of the study showed that trajectory prediction significantly improves interception accuracy, particularly when dealing with high-speed moving objects such as soccer balls.

Recent advances in deep learning have also improved object detection performance in robotic

vision systems. Bochkovskiy et al. [3] introduced the YOLOv4 object detection framework, which provides a balance between detection accuracy and computational efficiency. YOLO-based detection models are capable of identifying objects within images in real time, making them suitable for robotic perception tasks. For embedded systems with limited computational power, lightweight versions such as YOLOv4-Tiny can be deployed to detect objects such as balls or obstacles with relatively high speed. These models allow robotic systems to operate in more complex environments where traditional color-based detection methods may struggle due to variations in lighting conditions or background clutter.

Another approach for detecting objects in robotic systems is color-based segmentation combined with morphological operations. Sharma et al. [4] proposed a color-based object tracking method designed for mobile robots. In their work, images captured by a camera are converted from RGB to HSV color space to isolate objects with specific color ranges. The segmented regions are then processed using morphological operations such as erosion and dilation to remove noise and improve detection accuracy. Finally, contour analysis is used to determine the centroid of the detected object, which can then be used for robot navigation or object tracking tasks. This approach is computationally efficient and well suited for systems that require real-time processing using low-cost hardware platforms.

A specific application closely related to the current project is the implementation of robotic soccer goalkeepers. Al-Khalifa et al. [5] developed a vision-based robotic goalkeeper capable of detecting and intercepting a soccer ball using a camera and a servo-actuated arm. In their system, the camera continuously captures frames of the playing field, and the ball is detected using image segmentation techniques. The trajectory of the ball is estimated based on its motion between frames, allowing the system to predict where the ball will cross the goal line. A control algorithm then commands the robotic arm to move to the predicted interception point. Experimental results demonstrated that the system was capable of successfully blocking a significant percentage of shots, highlighting the potential of integrating computer vision with robotic control systems.

In general, most vision-based robotic systems follow a similar processing pipeline. First, images are acquired from a camera sensor connected to the processing unit, which may be a laptop, embedded computer, or microprocessor such as a Raspberry Pi. Next, image preprocessing techniques such as filtering and noise reduction are applied to enhance the quality of the captured images. Feature extraction methods are then used to identify relevant objects within the scene, such as balls, obstacles, or markers. Finally, control algorithms process the extracted features to generate commands for actuators such as motors or servos. This architecture allows robotic systems to convert visual information into physical actions, enabling interaction with the environment.

For embedded robotic systems, computational efficiency is an important design consideration. Platforms such as Raspberry Pi provide sufficient computational power for many com-

puter vision tasks while maintaining low cost and energy consumption. When combined with libraries such as OpenCV, these platforms can perform real-time image acquisition, filtering, segmentation, and object tracking. In addition, the Raspberry Pi can communicate with microcontrollers such as Arduino to control motors and actuators. This distributed architecture allows the Raspberry Pi to perform high-level decision making based on image processing results while the Arduino handles low-level motor control tasks.

Overall, the literature demonstrates that vision-based robotic goalkeeping systems rely heavily on robust object detection and trajectory prediction algorithms. Techniques such as color segmentation, contour detection, Kalman filtering, and deep learning-based object detection have been successfully applied in various robotic applications. By combining these approaches with embedded processing platforms and simple control mechanisms, it is possible to design an effective robotic goalkeeper capable of detecting, predicting, and intercepting a moving ball in real time.

Chapter 2

Hardware Design

2.1 Component List

Table 2.1: Hardware Components Required for the Goalkeeper Robotic Arm

Component Name	**Quantity**	**Approx. Price (EGP)**	**Purchase Source**
Raspberry Pi 4 Model B (4GB)	1	4750	RAM Electronics
Raspberry Pi Camera Module V2	1	800	RAM Electronics
Raspberry Pi Camera Cable	1	185	RAM Electronics
Arduino Uno R3	1	750	RAM Electronics
MicroSD Card 64GB	1	495	RAM Electronics
High Torque Servo Motor (FS5103B)	1	300	RAM Electronics
USB Cable (Arduino Connection)	1	40	RAM Electronics

2.2 Project Plan

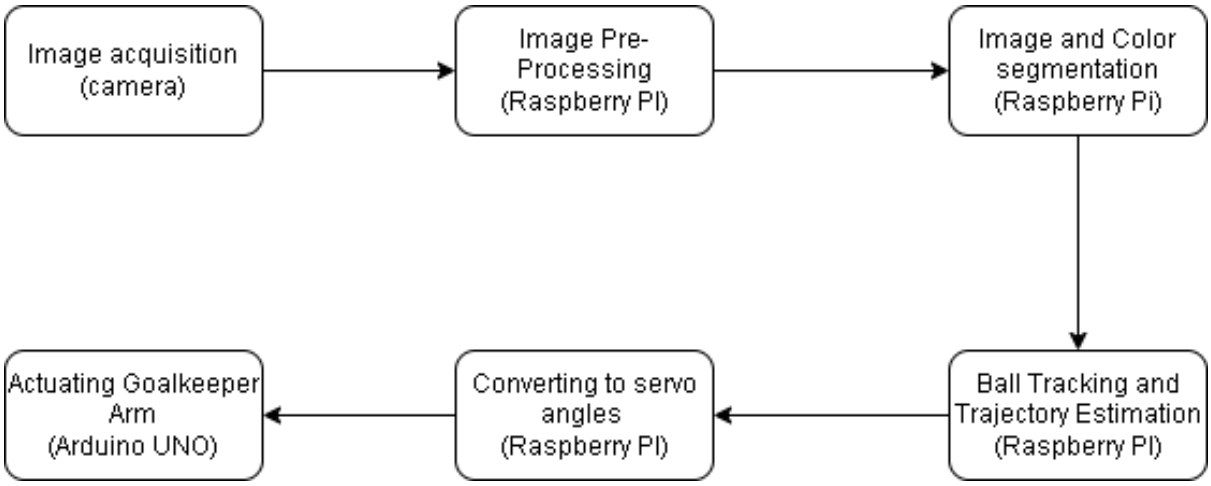


Figure 2.1: Project Plan

2.3 Solidworks Design

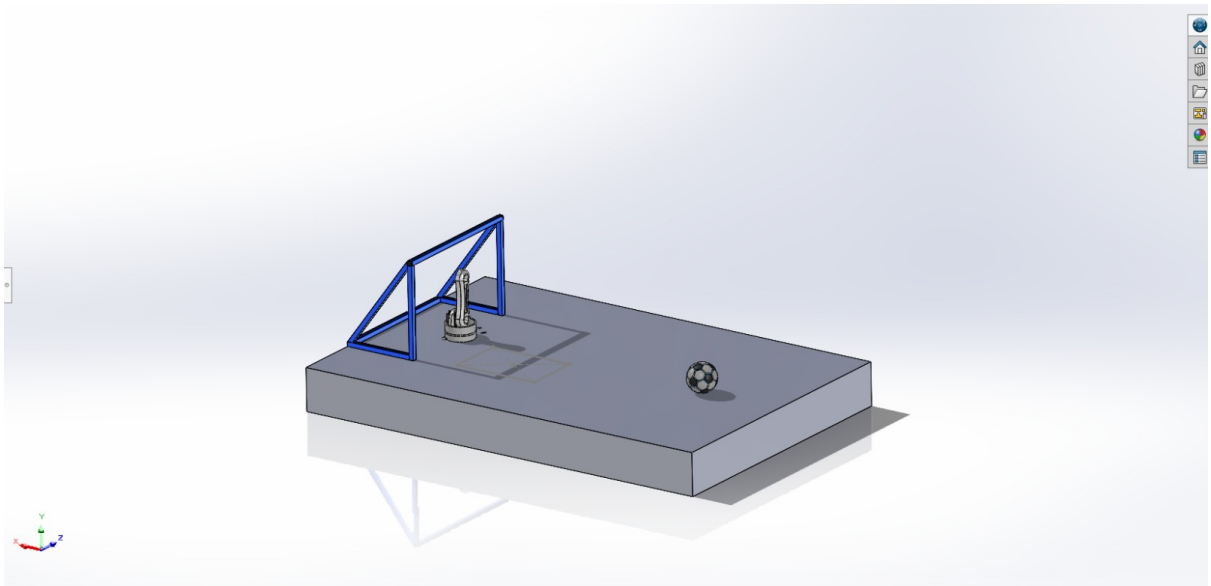


Figure 2.2: SolidWorks

Chapter 3

Circuit Design

For the circuits design the Raspberry Pi 4b will be the main controller with Camera connected to iy. And the Arduino UNO will be connected to RPI through the USB Cable. Finally, Servo motor will be connected to the Arduino UNO.

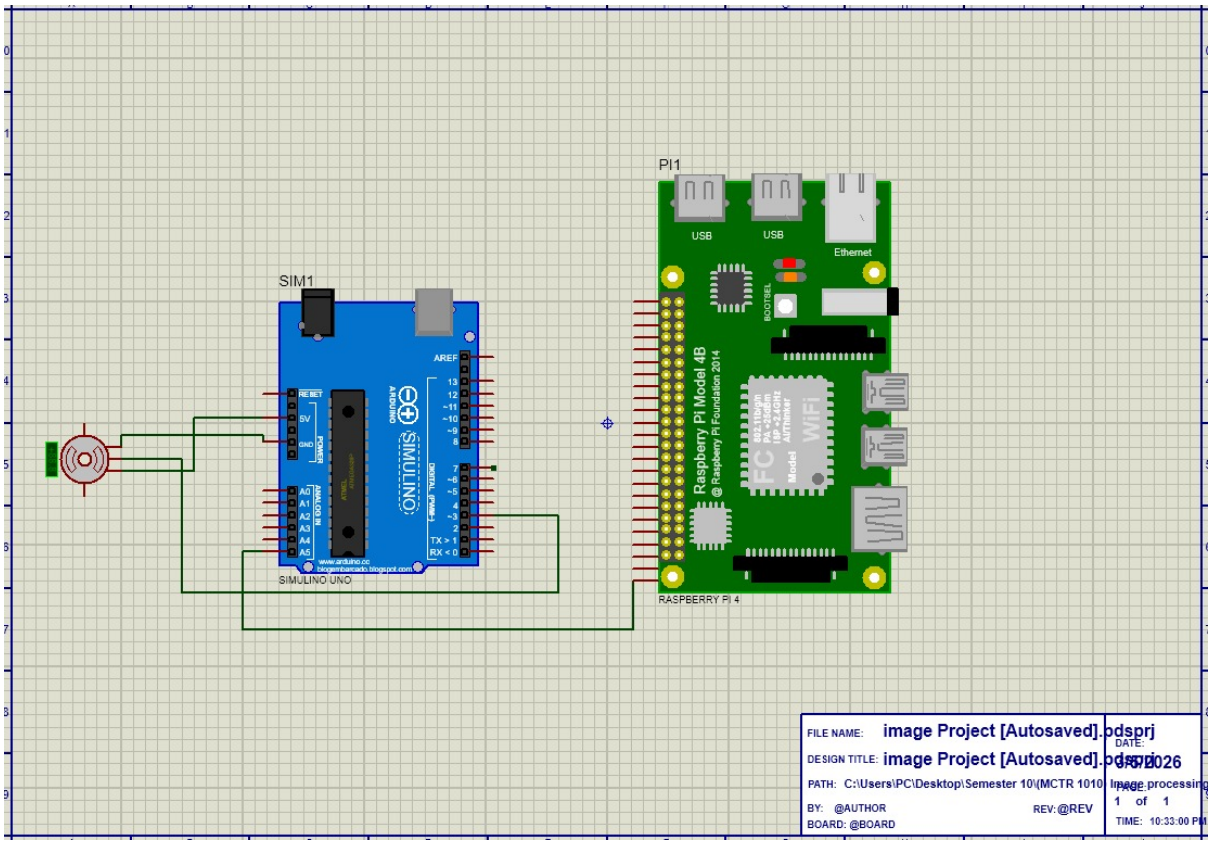


Figure 3.1: Circuits Design

Chapter 4

Raspberry Pi Capture

Figure 4.1: Image Captured by RPI Camera

Chapter 5

Image Processing and Motor Actuation

5.1 Image Feature Extraction

In order to control the movement of the robotic goalkeeper arm, relevant features must first be extracted from the captured images. The Raspberry Pi camera continuously captures frames of the playing field, which are then processed using computer vision techniques implemented in Python with the OpenCV library.

The primary feature extracted from each frame is the position of the ball within the image. The image is first converted from the RGB color space to the HSV color space, which allows easier segmentation of colored objects. A predefined color range corresponding to the color of the ball is applied in order to isolate the ball from the background.

After color segmentation, morphological operations such as erosion and dilation are applied to remove noise and improve the quality of the detected region. Contour detection is then performed on the segmented image in order to identify the boundaries of the ball. From the detected contour, the centroid of the ball is calculated, which represents the ball's position in the image frame.

The extracted centroid coordinates are used as the main image feature for the control system. These coordinates provide information about the horizontal position of the ball relative to the goal area, which is then used to determine the direction and speed of the servo motor that moves the robotic arm.

5.2 Motor Actuation Based on Image Features

In this project, image processing techniques are utilized to detect a colored ball and control the motion of a robotic goalkeeper arm. The system relies on extracting meaningful features from

the captured image to determine the position of the ball and accordingly actuate the motor.

5.2.1 Color-Based Ball Detection

The ball is detected using a color segmentation technique in the HSV color space. The captured RGB image is converted into HSV representation to improve robustness against lighting variations. A thresholding operation is then applied to isolate the yellow color of the ball.

The selected HSV range for yellow detection is defined as follows:

- Hue: 20 – 35
- Saturation: 100 – 255
- Value: 100 – 255

This process generates a binary mask where the detected ball appears in white while the rest of the image is suppressed.

5.2.2 Feature Extraction

After applying the color mask, contour detection is performed to identify the largest object corresponding to the ball. The centroid of the detected contour is calculated and used as the main feature for decision making.

5.2.3 Control Strategy

The horizontal position of the ball is divided into three regions:

- Left region
- Center region
- Right region

Depending on the position of the ball, the system sends a command to the Arduino to move the goalkeeper arm accordingly.

- If the ball is on the left $\rightarrow$ move left
- If the ball is on the right $\rightarrow$ move right
- If the ball is in the center $\rightarrow$ stay centered

Once the ball position is detected, the system generates control commands for the servo motor. The Raspberry Pi processes the image frames and determines the horizontal displacement of the ball relative to the center of the goal.

If the detected ball position lies to the left of the center, the servo motor is commanded to rotate in the left direction in order to intercept the ball. Similarly, if the ball is detected on the right side of the frame, the servo motor rotates in the right direction. When the ball is approximately aligned with the center region, the motor remains stationary in order to maintain the goalkeeper arm in the optimal blocking position.

The control signal generated by the Raspberry Pi is transmitted to the Arduino Uno through a USB serial connection. The Arduino then generates the appropriate Pulse Width Modulation (PWM) signal required to drive the servo motor. This architecture allows the Raspberry Pi to handle the computationally intensive image processing tasks while the Arduino performs the real-time motor control.

5.2.4 Visualization of Detection Results

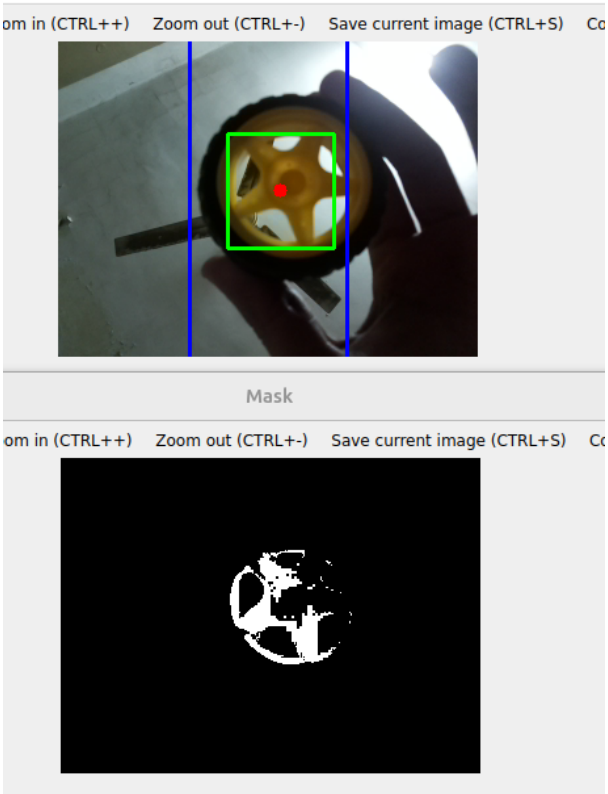


Figure 5.1: Ball in Center

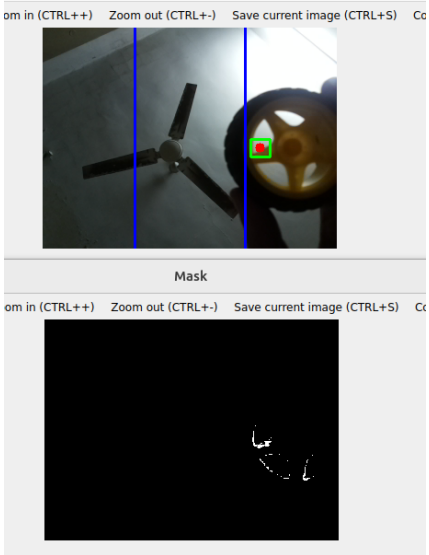


Figure 5.2: Ball on Right

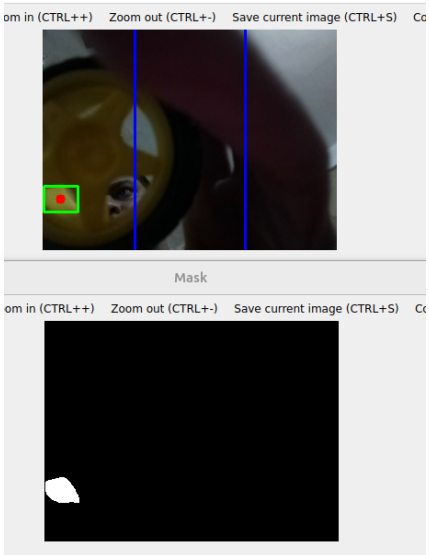


Figure 5.3: Ball on Left

5.3 Contrast Enhancement

Contrast enhancement is applied to improve the visibility of important features within the captured image before performing further image processing operations. In the raw image captured by the Raspberry Pi camera, variations in lighting conditions and shadows can reduce the clarity of objects in the scene. Increasing the contrast helps highlight edges and intensity differences, which simplifies later feature extraction steps.

The contrast operation works by adjusting the intensity values of the pixels in the image. Each pixel value is scaled relative to the overall brightness range of the image so that darker regions become darker and brighter regions become brighter. This increases the separation between different intensity levels and makes objects more distinguishable from the background.

In this project, the captured RGB image is first converted to a grayscale representation in order to simplify the processing and reduce computational load. After conversion to grayscale, a contrast enhancement operation is applied to stretch the pixel intensity distribution. This process increases the difference between low and high intensity values, which improves the visibility of objects and scene details.

Figure 5.4 illustrates the effect of the contrast operation. The first image shows the original frame captured by the camera. The second image shows the grayscale conversion of the frame. The third image shows the result after applying the contrast enhancement, where the intensity differences in the image become more pronounced, making important features easier to detect in subsequent processing stages.

Figure 5.4: Contrast enhancement process showing the original image, grayscale image, and contrast enhanced image

5.4 Image Rotation

Image rotation is applied to correct the orientation of the captured frames before performing further image processing operations. Depending on the physical mounting position of the Rasp-

berry Pi camera, the captured image may appear tilted or rotated relative to the actual scene. This misalignment can affect the accuracy of feature detection and object tracking. Therefore, a rotation operation is applied to align the image with the expected reference frame.

The rotation operation works by transforming the coordinates of each pixel in the image according to a predefined rotation angle. This transformation rotates the entire image around its center while preserving the spatial relationships between pixels. In practice, this is performed using a rotation matrix that maps the original pixel positions to their new rotated positions.

In the implemented system, the captured frame is rotated by a fixed angle to correct the camera orientation. This ensures that the horizontal and vertical directions in the image correspond correctly to the physical movement directions of the robotic arm. By aligning the image with the coordinate system of the control algorithm, the detected object position can be accurately translated into motor commands.

Figure 5.5 illustrates the rotation process. The first image shows the original frame captured by the Raspberry Pi camera. The second image shows the rotated version of the frame after applying the rotation operation. The corrected orientation allows the system to process the image in a consistent coordinate reference, improving the reliability of subsequent feature extraction and control steps.

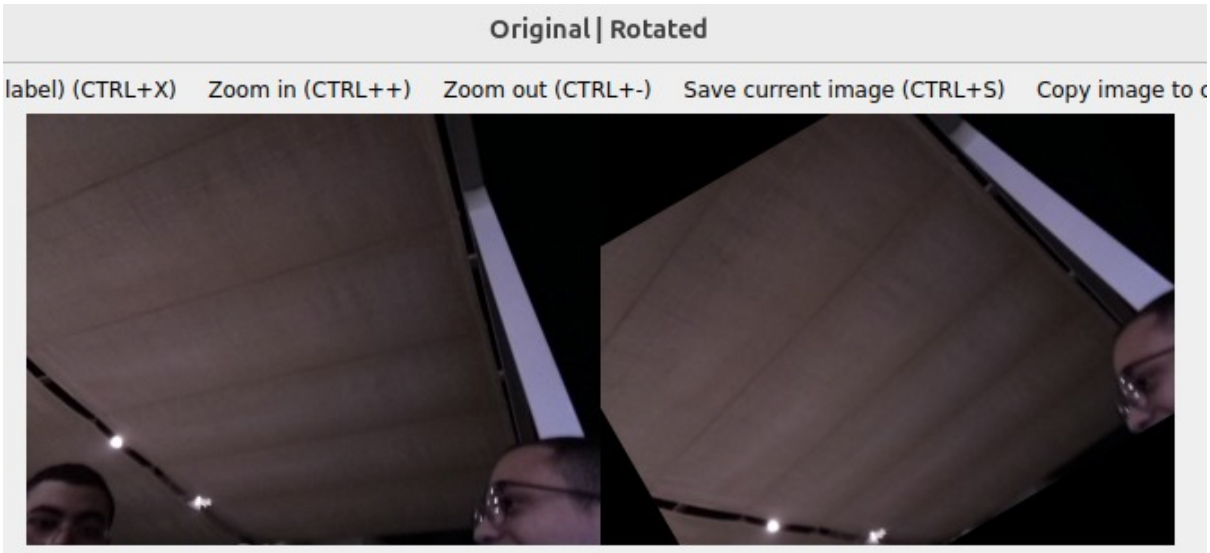


Figure 5.5: Image rotation process showing the original captured frame and the rotated image

5.5 Grayscale Conversion

Grayscale conversion is performed as an early preprocessing step in the image processing pipeline. The images captured by the Raspberry Pi camera are originally represented in the

RGB color space, where each pixel contains three color channels: red, green, and blue. While this representation provides full color information, it also increases the computational complexity of subsequent image processing operations.

To simplify the processing and reduce the computational load, the RGB image is converted into a grayscale image. In a grayscale image, each pixel is represented by a single intensity value instead of three color channels. This intensity value corresponds to the brightness of the pixel and is typically calculated as a weighted combination of the red, green, and blue components.

By converting the image to grayscale, the system focuses only on intensity variations within the image rather than color information. This simplification allows faster processing while still preserving important structural details such as edges, shapes, and object boundaries. These features are essential for detecting and tracking objects within the scene.

Figure 5.6 illustrates the grayscale conversion process. The first image shows the original RGB frame captured by the camera, while the second image shows the grayscale version of the same frame. The grayscale representation retains the structural information of the scene while reducing the data complexity, making it more suitable for further image processing steps.

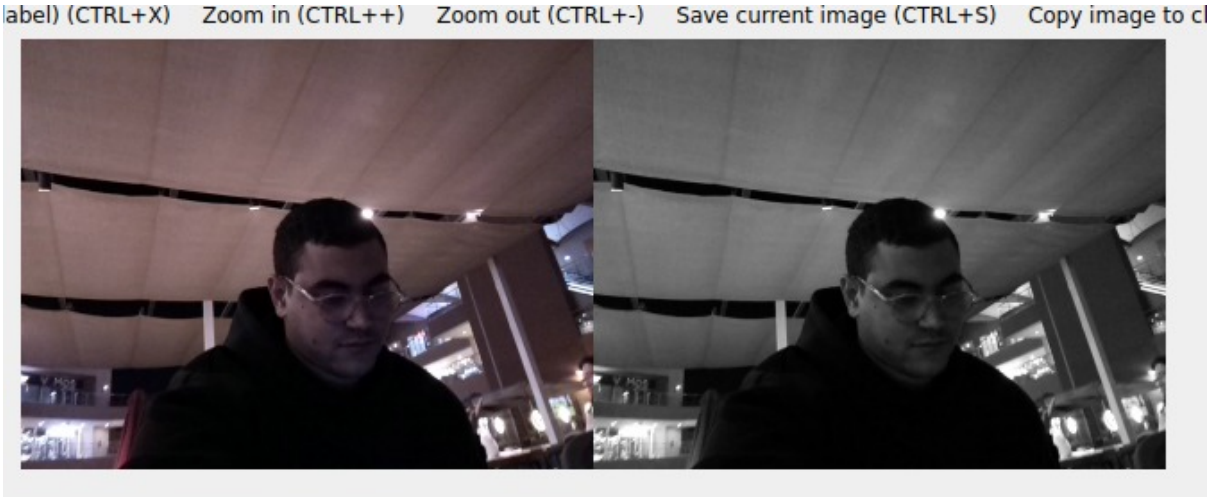


Figure 5.6: Grayscale conversion showing the original RGB image and the grayscale representation

5.6 Image Scaling

Image scaling is used to resize the captured frames in order to reduce the computational requirements of the image processing pipeline. High-resolution images contain a large number of pixels, which increases the processing time for operations such as filtering, feature extraction, and object detection. By reducing the image size, the system can process frames more

quickly while still retaining the essential visual information needed for the task.

The scaling operation works by resizing the image to a smaller resolution using interpolation techniques. During this process, the pixel values of the resized image are computed based on the values of neighboring pixels in the original image. This ensures that the visual structure of the image is preserved even after the resolution is reduced.

In this project, the captured frames are scaled down to a lower resolution before further processing steps are applied. This significantly reduces the computational load on the Raspberry Pi and allows the system to process frames at a higher rate, which is important for real-time operation of the robotic goalkeeper system.

Figure 5.7 illustrates the scaling process. The first image shows the original high-resolution frame captured by the camera, while the second image shows the resized version after applying the scaling operation. Despite the reduction in resolution, the main visual features remain clearly visible, allowing the system to continue processing the image effectively.

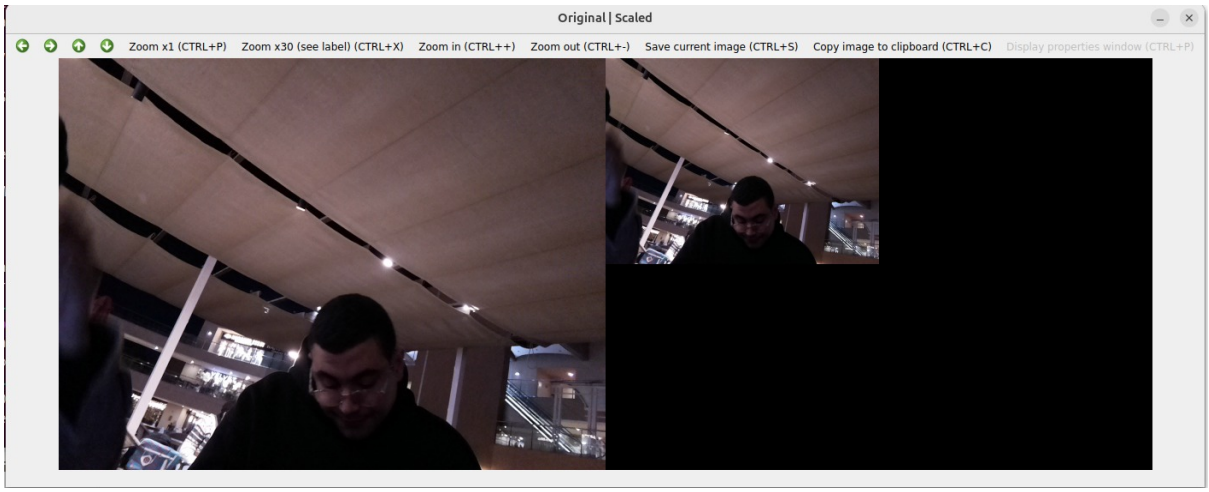


Figure 5.7: Image scaling showing the original captured frame and the resized image used for processing

5.7 Performance Comparison Between Raspberry Pi and Laptop

In this experiment, the same image processing algorithms were executed on both the Raspberry Pi and a personal laptop in order to evaluate the system performance. The comparison was conducted based on execution time, detection accuracy, and system responsiveness.

5.7.1 Execution Time

The processing time per frame on the Raspberry Pi was observed to be slightly higher than that on the laptop due to the lower computational capabilities of the embedded system. However, the difference was not significant for the implemented algorithms, as the system was still capable of operating in near real-time.

5.7.2 Detection Accuracy

The accuracy of ball detection was found to be nearly identical on both platforms. Since the same OpenCV-based algorithms and parameters were used, both systems produced consistent results in detecting the ball and determining its position.

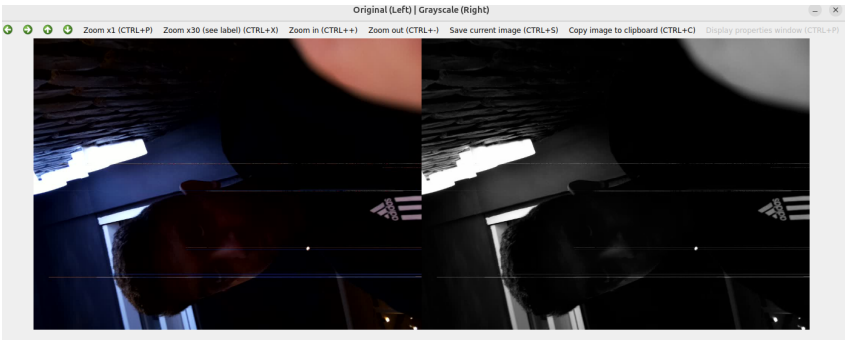
5.7.3 System Responsiveness

Although the laptop exhibited faster processing and smoother visualization, the Raspberry Pi demonstrated sufficient responsiveness for the application. The delay observed on the Raspberry Pi was minimal and did not affect the functionality of the closed-loop control system.

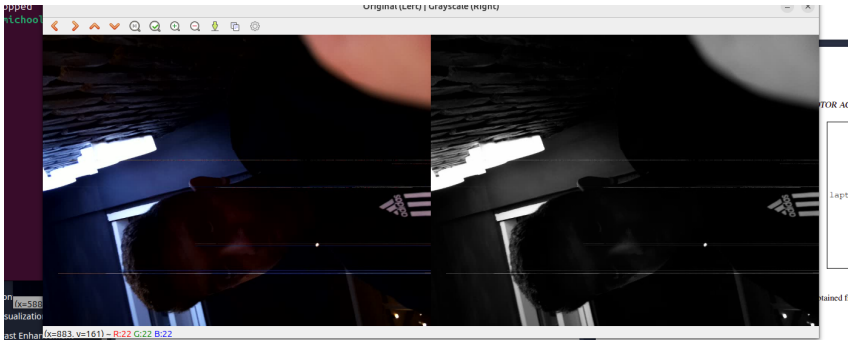
5.7.4 Discussion

Overall, the Raspberry Pi proved to be an effective platform for implementing the required image processing tasks. Despite its lower computational power compared to the laptop, it was able to achieve comparable performance in terms of detection accuracy and control functionality. Therefore, the Raspberry Pi is considered suitable for real-time embedded applications such as the goalkeeper robotic system.

-



(a) Raspberry Pi Output



(b) Laptop Output

Figure 5.8: Comparison of the detection results obtained from the Raspberry Pi and the laptop.

5.8 Integrated System Implementation and Results

5.8.1 Overall System Methodology

The implemented system follows a structured methodology that integrates image processing with real-time hardware control. As illustrated previously in the project plan (Figure 2.1), the system operates as a closed-loop pipeline starting from image acquisition and ending with motor actuation.

The methodology can be summarized as follows:

- Capture real-time images using the Raspberry Pi camera.
- Apply preprocessing techniques such as scaling, rotation, and contrast enhancement.
- Perform color-based segmentation to isolate the ball.
- Extract features such as the centroid position of the ball.
- Map the detected position into predefined zones.

- Generate control commands based on the detected zone.
- Actuate the servo motor via Arduino to intercept the ball.

This structured pipeline ensures continuous feedback between the environment and the robotic system, enabling real-time response.

5.8.2 Detailed Implementation on Hardware

The implementation of the system was carried out on the fabricated hardware setup described in Chapter 2 and Chapter 3. The Raspberry Pi serves as the main processing unit, executing all image processing operations using Python and OpenCV.

The camera continuously captures frames, which are processed sequentially through the pipeline. The processed output includes a binary mask and a detected contour representing the ball.

Once the centroid of the ball is calculated, its position is mapped relative to the frame width. This position is then transmitted to the Arduino UNO via serial communication.

The Arduino interprets the received data and generates PWM signals to control the servo motor. The motor rotates accordingly to align the robotic arm with the predicted ball position.

This division of tasks ensures efficient system operation, where the Raspberry Pi handles computation while the Arduino handles real-time actuation.

5.8.3 Experimental Results

The results of the implemented image processing pipeline are shown in Figure 5.9.

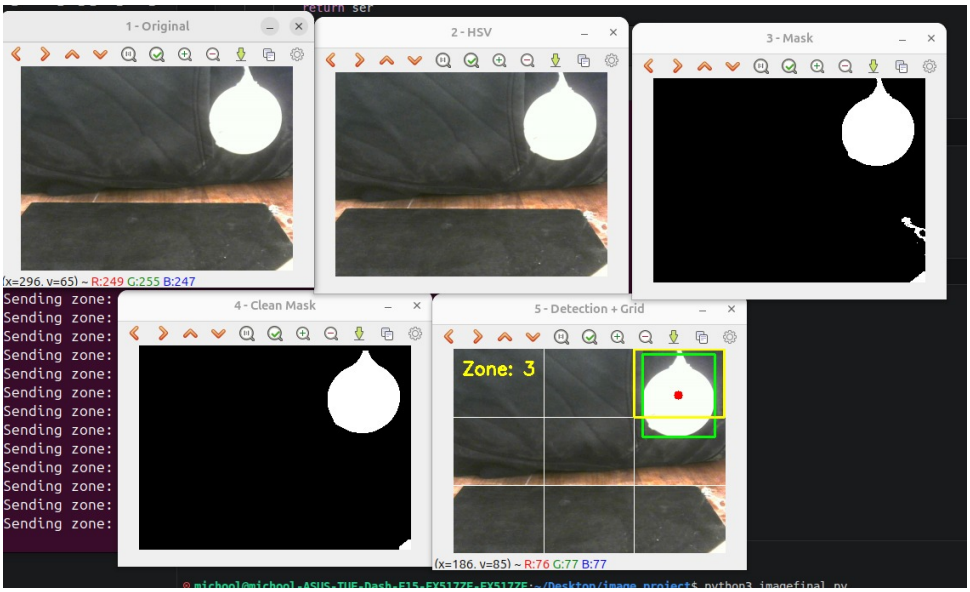


Figure 5.9: Image processing pipeline showing original image, HSV conversion, masking, noise removal, and final detection with grid

As shown in the figure, the system successfully performs the following:

- The original image is captured under real-world lighting conditions.
- The HSV conversion enhances color separation.
- The binary mask isolates the ball effectively.
- Morphological processing removes noise and improves object clarity.
- The final detection identifies the ball and marks its centroid.

The grid overlay divides the frame into multiple regions, allowing the system to determine the exact zone of the ball. In the shown example, the system correctly detects the ball in Zone 3 and outputs the corresponding control command.

5.8.4 Hardware Setup and System Operation

The complete hardware setup used in the experiment is shown in Figure 5.10.

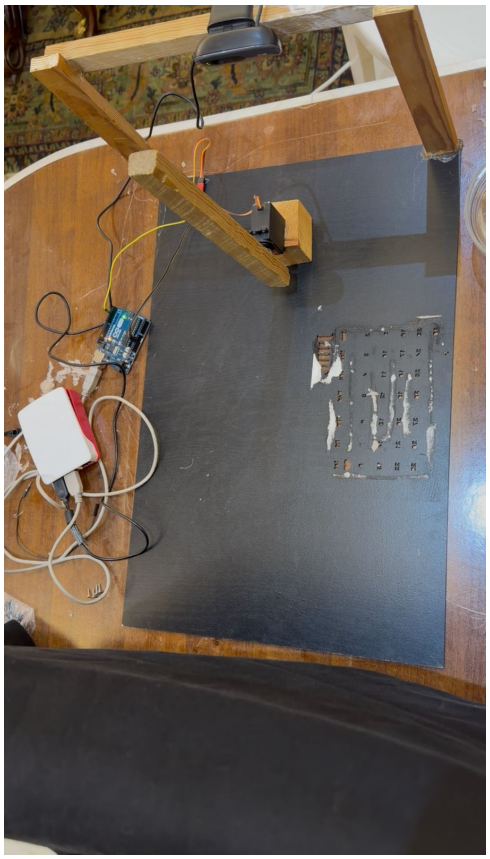


Figure 5.10: Complete Hardware Setup of the Goalkeeper Robotic Arm System

The system operates as follows:

- The camera captures the scene and sends frames to the Raspberry Pi.
- The Raspberry Pi processes each frame and extracts the ball position.
- The computed position is transmitted to the Arduino via USB serial communication.
- The Arduino converts the received signal into PWM signals.
- The servo motor actuates the robotic arm accordingly.

This closed-loop interaction ensures that the robotic arm continuously adjusts its position based on the detected ball location.

5.8.5 Discussion of Results

The implemented system demonstrates reliable performance in detecting the ball and generating appropriate control signals. The centroid detection (represented by the red point) accurately indicates the position of the ball within the frame.

The use of HSV color segmentation proved effective in isolating the object even in the presence of background variations. Additionally, morphological operations significantly improved the quality of detection by eliminating noise.

However, some limitations were observed:

- Sensitivity to lighting variations affects segmentation accuracy.
- Objects with similar colors may introduce detection errors.
- Processing delay on the Raspberry Pi slightly affects responsiveness.

Despite these limitations, the system achieves real-time performance and demonstrates the feasibility of implementing image-based control on embedded platforms.

5.8.6 Summary

This section presented the complete integration of the image processing pipeline with the hardware system. The results confirm that the system can successfully detect the ball, determine its position, and actuate the robotic arm accordingly in a closed-loop manner.

5.9 Code Functionality and Processing Logic

The implemented image processing system is based on a structured sequence of operations executed in real time using Python and the OpenCV library. The code is designed to continuously capture frames from the Raspberry Pi camera, process each frame, and generate control signals based on the detected object position.

At the beginning of execution, the camera is initialized and configured to capture frames at a fixed resolution. A continuous loop is then established to process each incoming frame sequentially.

For each frame, the following operations are performed:

- **Frame Acquisition:** The system reads a frame from the camera stream. This frame represents the current state of the environment.
- **Preprocessing:** The captured frame is optionally resized to reduce computational load. In addition, rotation is applied if required to align the image with the physical orientation of the system.
- **Color Space Conversion:** The RGB image is converted into the HSV color space. This allows easier isolation of the ball based on its color properties.

- **Thresholding:** A predefined HSV range is applied to generate a binary mask. Pixels within the selected range are assigned a value of 1 (white), while all other pixels are set to 0 (black).
- **Noise Filtering:** Morphological operations such as erosion and dilation are applied to remove small noise regions and improve the quality of the detected object.
- **Contour Detection:** The processed mask is analyzed to detect contours. The largest contour is selected as the most likely representation of the ball.
- **Centroid Calculation:** Image moments are used to compute the centroid of the detected contour. This centroid represents the position of the ball in the frame.
- **Zone Classification:** The horizontal coordinate of the centroid is compared against predefined thresholds to determine whether the ball lies in the left, center, or right region of the frame.
- **Command Generation:** Based on the detected zone, a corresponding command is generated (e.g., move left, move right, or remain centered).
- **Serial Communication:** The generated command is transmitted from the Raspberry Pi to the Arduino UNO through a serial connection.
- **Loop Execution:** The process repeats continuously for each incoming frame, enabling real-time tracking and response.

This implementation ensures that the system operates in a continuous feedback loop, where each processed frame directly influences the movement of the robotic arm. The use of efficient operations such as color thresholding and contour detection allows the system to maintain real-time performance on the Raspberry Pi.

5.10 Actuation Based on Visual Detection

Once the ball is successfully detected in the processed image, the system performs actuation by translating the visual information into motor movement. The actuation process is based on the position of the detected white ball within the image frame.

After generating the binary mask, the ball appears as a white region against a black background. The contour detection stage identifies this white region and calculates its centroid, which represents the exact position of the ball.

The horizontal coordinate of the centroid is then used to determine the relative position of the ball with respect to the center of the image frame. This position is mapped into predefined zones that correspond to different motor actions.

In order to improve the accuracy of position classification, the image frame is divided into a grid consisting of nine regions (3×3). Each region represents a specific spatial location of the ball within the field of view. The grid is organized as follows:

- Region 1: Top-left corner
- Region 2: Top-center
- Region 3: Top-right corner
- Region 4: Middle-left
- Region 5: Center
- Region 6: Middle-right
- Region 7: Bottom-left corner
- Region 8: Bottom-center
- Region 9: Bottom-right corner

The centroid of the detected ball is mapped to one of these regions based on its coordinates. This grid-based approach provides a more structured representation of the ball position while improving the reliability of the detection process.

The actuation logic operates as follows:

- If the centroid of the white ball lies on the **left side** of the frame, the system generates a command to rotate the servo motor to the left.
- If the centroid lies on the **right side** of the frame, the system generates a command to rotate the servo motor to the right.
- If the centroid is located near the **center region**, the servo motor maintains its current position to block the ball.

This process effectively enables the system to ”see” the white ball and respond by aligning the robotic arm with its position.

The generated command is transmitted from the Raspberry Pi to the Arduino UNO through serial communication. The Arduino then converts this command into a Pulse Width Modulation (PWM) signal, which controls the angular position of the servo motor.

As a result, the robotic arm continuously adjusts its position to track the detected ball in real time. This creates a closed-loop system where visual feedback directly controls mechanical motion.

5.10.1 Real-Time Tracking Behavior

The continuous execution of the image processing loop ensures that the system updates the ball position at every frame. As the ball moves across the field of view, the centroid position changes accordingly.

The system reacts dynamically to this change:

- When the ball moves gradually, the servo motor follows smoothly.
- When the ball moves rapidly, the system updates its position in discrete steps based on the detected zone.

This behavior allows the robotic goalkeeper arm to track the motion of the ball and position itself to intercept it.

5.10.2 Limitations in Actuation

Although the system successfully tracks the ball, some limitations were observed:

- **Discrete Movement:** The use of zone-based control results in step-wise movement rather than smooth continuous tracking.
- **Delay in Response:** Processing time and communication delay introduce slight lag between detection and actuation.
- **Servo Speed Constraints:** The physical speed of the servo motor limits the system's ability to react to very fast ball movements.

Despite these limitations, the system demonstrates effective real-time tracking and actuation based on visual input.

Bibliography

[1] S. Kim, J. Lee, and H. Park. Real-time ball tracking using computer vision for robotic applications. *IEEE Access*, 9:11234–11245, 2021.

[2] J. Liu, T. Wang, and Y. Zhang. Vision-based ball trajectory prediction for robotic goalkeepers. *Robotics and Autonomous Systems*, 150:103987, 2022.

[3] A. Bochkovskiy, C. Wang, and H. M. Liao. Yolov4: Optimal speed and accuracy of object detection. *Computer Vision and Pattern Recognition*, 2021.

[4] P. Sharma, R. Gupta, and K. Singh. Color-based object detection and tracking using opencv for mobile robots. *Sensors*, 23(8):4125, 2023.

[5] M. Al-Khalifa, M. Rahman, and S. Ahmed. Design and implementation of a vision-based soccer goalkeeper robot. *IEEE Access*, 10:34567–34578, 2022.